Introduction to Software Engineering

Lecture 7:
Design – Part I

Outline

1. What is Design?
2. Software Design Model and Views
3. Software Design Principles
4. Important Remarks
5. Summary

Outline

1. What is Design?
2. Software Design Model and Views
3. Software Design Principles
4. Important Remarks
5. Summary

What is Design?

In general terms (not within the software context):

- As a noun: design is a meaningful engineering representation (i.e. a model) of something that is to be built.

- As a process: software design is an iterative process through which requirements are translated into a "blueprint" for constructing the something.

Software Design

In this class we will focus only on the design model

- As a noun: It is a model that guides the conversion of a specification into executable code, i.e. the means by which a problem description is turned into a problem solution.
- As a process: It is a process that involves a sequence of steps that enable the designer to describe all aspects of the software to be built.

Outline

1. What is Design?
2. Software Design Model and Views
3. Software Design Concepts/Principles
4. Important Remarks
5. Summary

Software Design Model

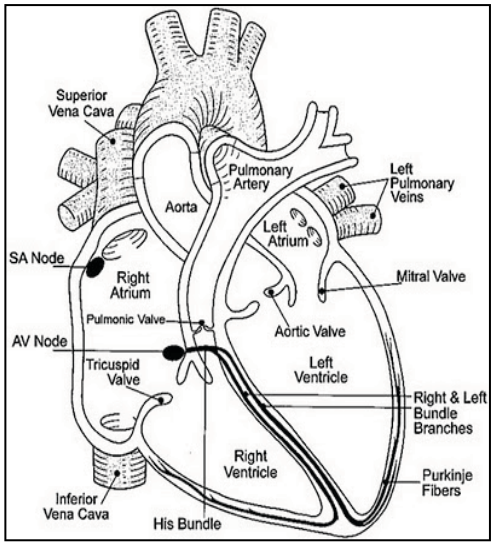
- It is the equivalent of an architect's plans for a house.

  It begins by representing the totality of the thing to be built and slowly refines the thing to provide guidance for constructing each detail.

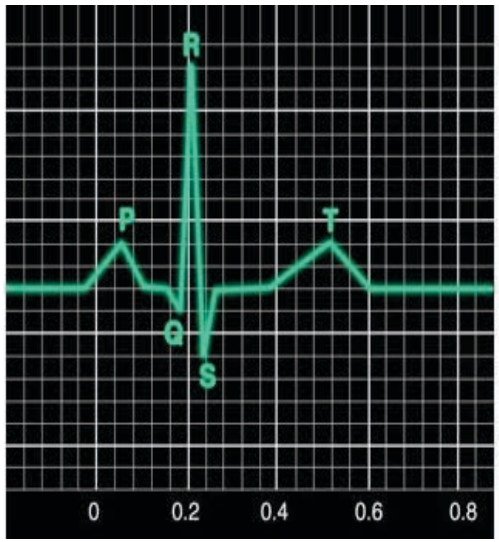
- The design model that is created for software provides a variety of different views.

Design Model Views

The human body as a system



This a stactic view of one of its subsystems



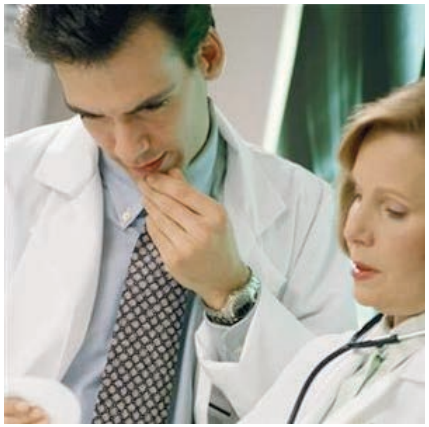
This a dynamic view of the same subsystem

Design Model Views

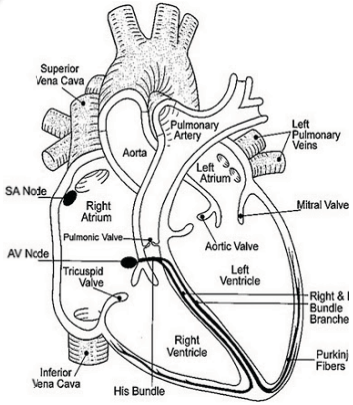
Different stakeholders have different interest on the system and therefore are interested in different parts and views

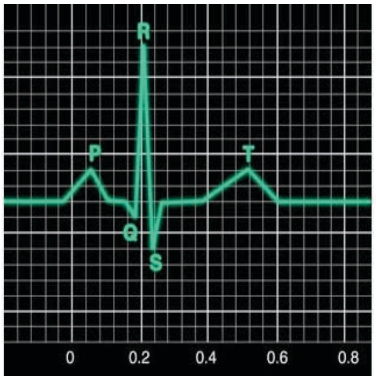
Design Model Views

These views are good for the cardiologists



… but are not for the orthopaedist





Design Model Views

- Static Views: it shows structures that are static including code-oriented structures like modules, classes, database table structures, file and data structures, layers, and so forth.
- Dynamic Views: it shows structures that are apparent at runtime and tend to be the loci of computation such as processes, threads, bjects, and so forth.

Design Model Views

- Physical Views: In the physical perspective we will often see the "real stuff" of the system, such as computers, networks, routers, mechanical systems, sensors, wiring/cabling, and so forth.

Which kinds of stakeholders are related to these views?

Outline

1. What is Design?
2. Software Design Model and Views
3. Software Design Principles
4. Important Remarks
5. Summary

What Makes a Good Design?

- The design must consider all of the explicit requirements collected during the Requirements Analysis Phase.
- The design must be a readable, understandable model that guides those who generate code and those who test and subsequently maintain the software.

Design Principles

- Software design principles represent a set of guidelines that helps us to avoid having a bad design.
- There are many design principles that have become best practices over the years.

Design Principles

Some very well known design principles are:

- Modularity
- Cohesion
- Coupling
- Information Hiding/Encapsulation

Design Principles

Modularity

- Modularity in software design is an approach that subdivides a system into smaller parts, called modules, that can be independently created and then used in different systems to drive multiple functionalities.
- It is easier to adopt a divide and conquer approach when designing software.

Design Principles

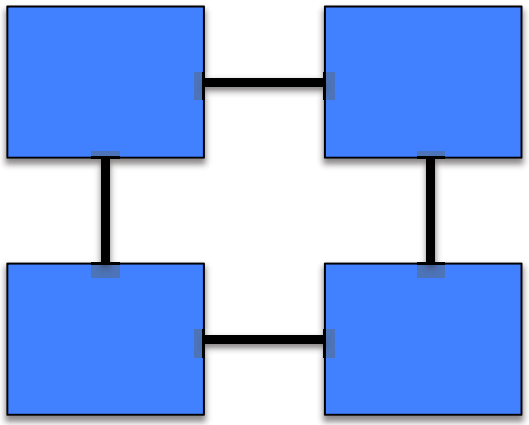
Cohesion complements modularity.
We always want **high cohesion**.

Cohesion

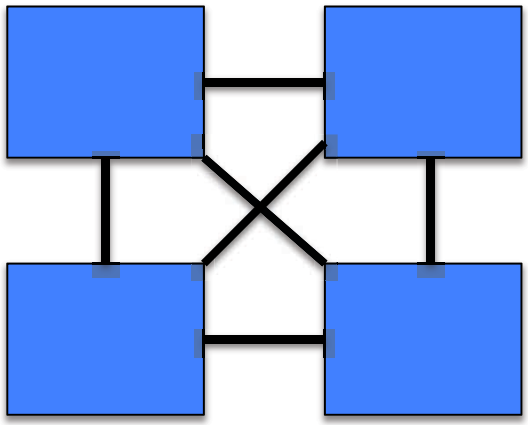
- A cohesive module performs a single task requiring little interaction with procedures being performed by other modules of a program.
- Stated simply, a cohesive module should (ideally) perform just one function or perform a family of related functions.

Design Principles

Coupling: Degree of Dependency



Low Coupling



High Coupling

Design Principles

We always want **low coupling**.

Coupling

- It is the degree to which a module relies on other modules.
- Coupling is usually contrasted with cohesion. Low coupling often correlates with high cohesion, and vice versa.
- High coupling is bad for modifiability as it makes hard the replacement of modules without affecting others.

Design Principles

Information Hiding/Encapsulation (1/3)

- Modules should be designed so that their information/implementation is inaccessible to other modules that have no need for such information.
- This principle is about segregating the design decisions in a module that are most likely to change (e.g. the details about particular algorithm used by a module).

Design Principles

Information Hiding/Encapsulation (2/3)

- It helps to protect parts of the module from extensive modification if its design is changed.
- The protection involves providing a stable interface which protects the remainder of the module from the implementation (the details that are most likely to change).

Design Principles

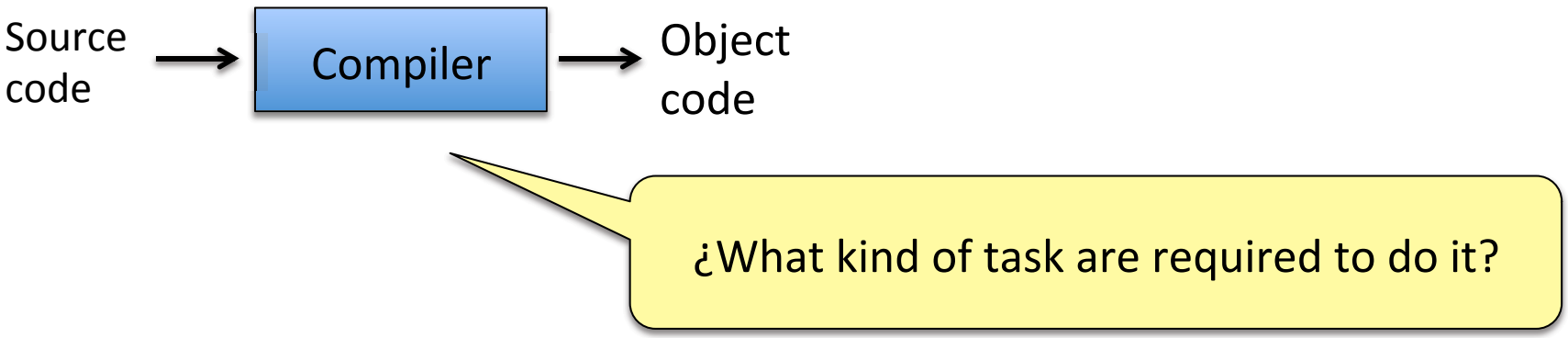
Information Hiding/Encapsulation (3/3)

- In this context, an interface is concept that refers to a point of interaction between modules.

Example

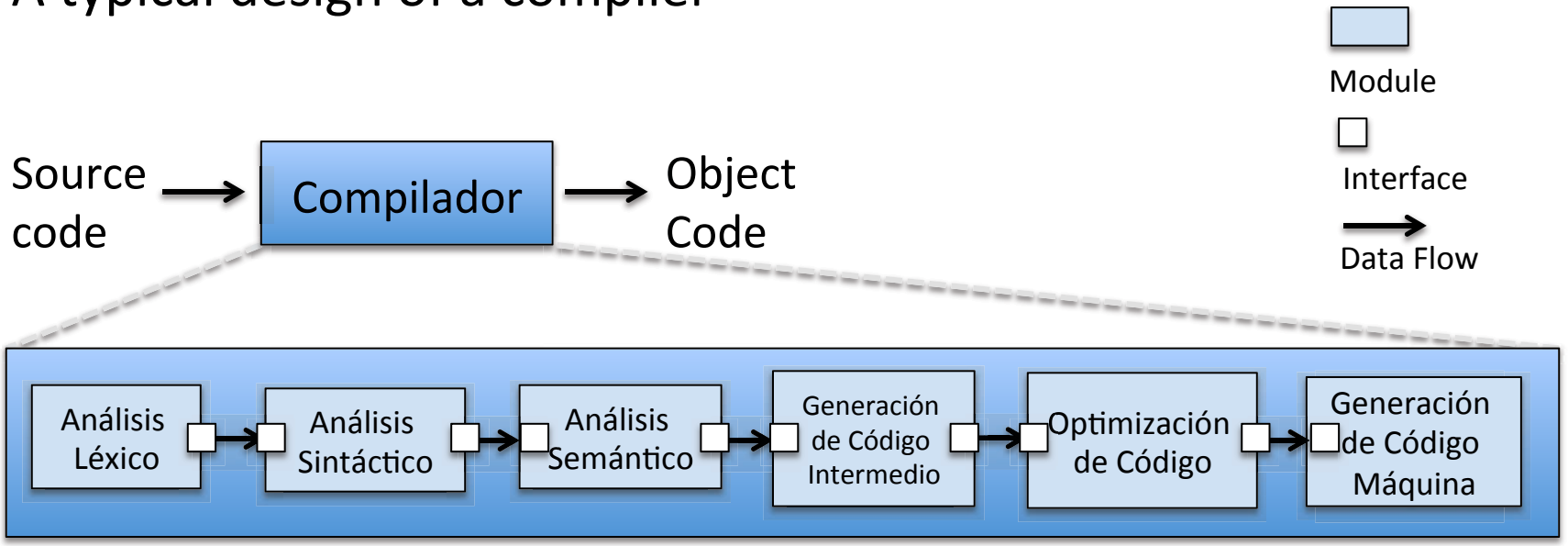
What is a compiler?

A **compiler** is a computer program that transforms source code written in a programming language into another computer language (the *target language*, often having a binary form known as *object code*).



Example

A typical design of a compiler

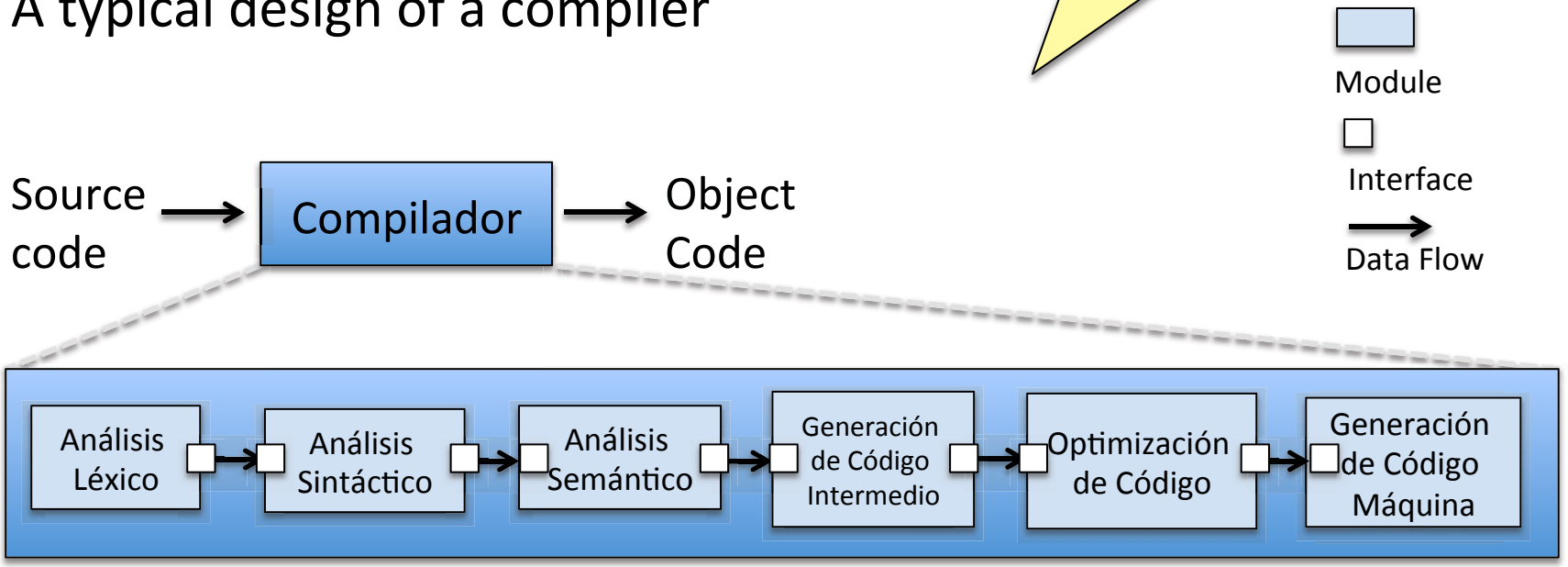


Example

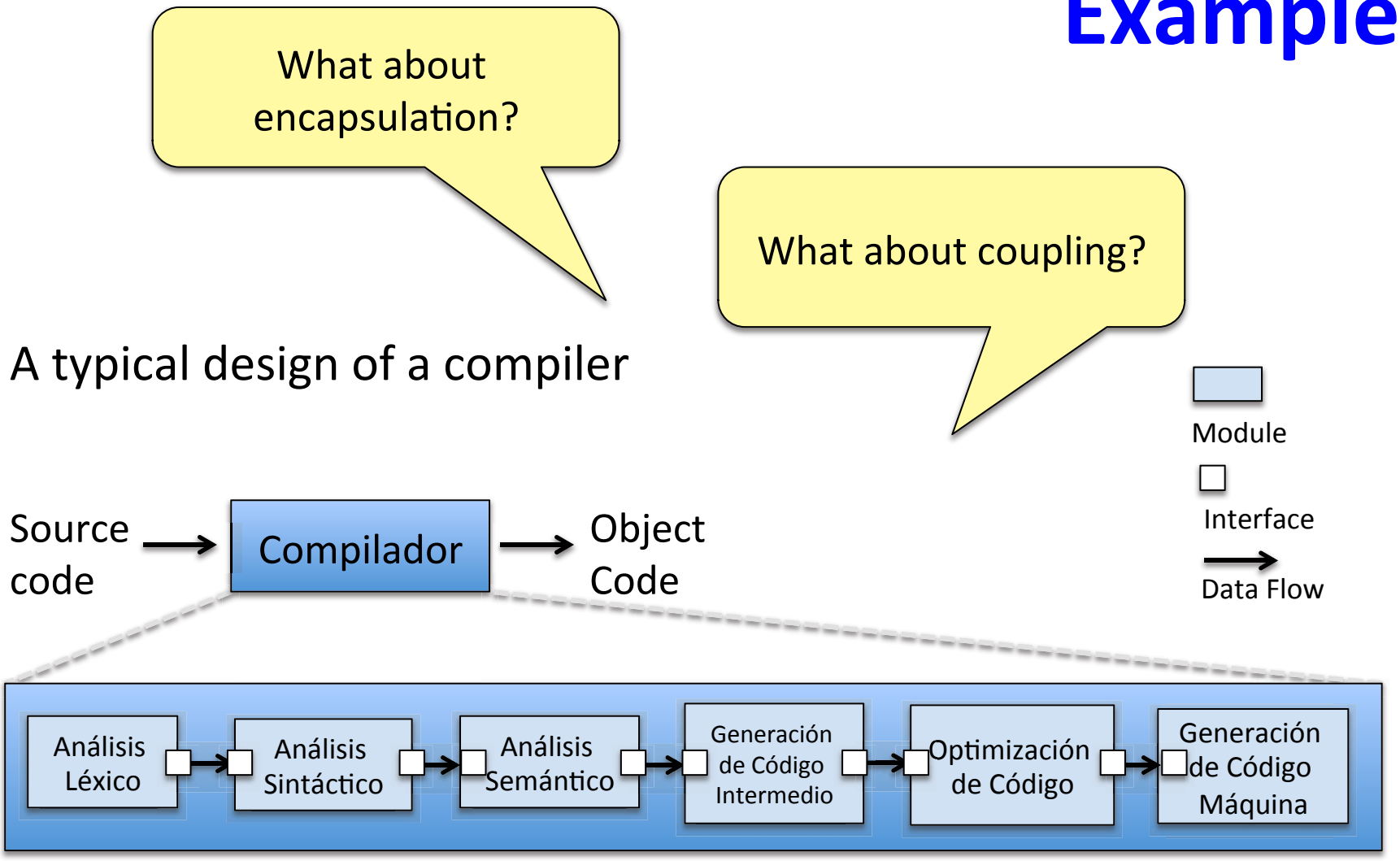
Can you recognize modularity here?

Can you recognize cohesion here?

A typical design of a compiler



Example


What about encapsulation?
What about coupling?
A typical design of a compiler
Module
Interface
Data Flow
Source code
Compilador
Object Code
Análisis Léxico
Análisis Sintáctico
Análisis Semántico
Generación de Código Intermedio
Optimización de Código
Generación de Código Máquina

Outline

1. What is Design?
2. Software Design Model and Views
3. Software Design Principles
4. Important Remarks
5. Summary

Important Remarks

- Unrelated parts of the system should be separated, and related parts of the system should be grouped together.
- Access to a grouped component should be through a simple interface; normally a small group of functions, each with a few parameters.
- We say that the details of a component are hidden by the interface.

Important Remarks

Separating unrelated modules:

- decreases the chance that modifying one module will adversely impact the other.
- makes it easier to reason about the individual modules because there is less to look at when separated.
- makes it is easier to reuse separated modules because it is possible to access the functionality of one without unnecessarily accessing the functionality of the other.

Important Remarks

- If the implementation of each module is appropriately hidden by its interface, then changing the implementation or extending its functionality should not change the existing interface.
- Changing an existing interface is problematic because all clients of the interface must be changed as well.

Important Remarks

- Successfully isolating a group of parts in a module depends on choosing a group of elements that can be elegantly presented to other modules with a concise, easy-to-understand interface.

Important Remarks

- The software design process is not simply a cookbook. Creative skill, past experience, a sense of what makes "good" software, and an overall commitment to quality are critical success factors for a competent design.

Important Remarks

- You should remember that design principles are just that principles. Sometimes is hard to find the correct balance among them. For example,
  - How do you know that you have divided the problem into the right number of modules? Basically by trial and error, although we have some intuitions mostly based on our experience.

Outline

1. What is Design?
2. Software Design Model and Views
3. Software Design Principles
4. Important Remarks
5. Summary

Summary

- What is Design?
- Software Design Model and Views
- Software Design Principles

Questions?

Comments?